

Oficina 1 — Um pino, dois significados

Do painel de atuadores à melodia no buzzer · PIC18F4550 · 16 MHz

Microcontroladores — DENE/UFMT

SEM NOTA

Esta oficina **não vale nota**. Ela existe para que os erros aconteçam aqui, e não no primeiro roteiro avaliado — e para que vocês toquem nos atuadores do projeto antes que qualquer um deles precise ser controlado. Quebre o programa de propósito, meça errado, recompile. Traga o registro preenchido: ele será discutido em sala, não corrigido.

NOTA

Continuidade com a aula de hoje. A aula terminou com uma ideia: a arquitetura decide coisas por vocês, e o programa tem de caber nelas. Aqui isso deixa de ser afirmação e passa a ser observável.

Os conceitos usados nesta sessão foram vistos há trinta minutos e não serão reexplicados: $T_{cy} = 4/f_{osc}$ e o núcleo a 16 MHz (§2 da aula), os registradores TRIS / LAT / PORT e a ordem LAT antes de TRIS (§4.1), e o bootloader como dono das palavras de configuração (§2.2).

OBJETIVOS

- Reconhecer que atuadores diferentes compartilham o mesmo pino, e configurar as chaves DIP que decidem qual deles está conectado.
- Estabelecer, por observação, que **frequência e proporção** de um sinal quadrado controlam grandezas distintas — e que o mesmo sinal significa coisas diferentes conforme o que está ligado ao pino.
- Verificar experimentalmente por que LAT é escrito antes de TRIS.
- Escrever, em C, as três funções que transformam um pino digital em um gerador de notas — e reconhecer os erros de C que não produzem mensagem nenhuma, apenas comportamento errado.
- Medir experimentalmente o custo em tempo de um laço, em vez de supô-lo.
- Apontar três limitações concretas de gerar tempo por contagem de instruções.

BANCADA

Roteiro da sessão. A oficina tem três partes e dois anexos. Se o tempo apertar, o corte é nos anexos — nunca no meio da Parte III, que só fecha quando a melodia toca.

Parte	Seções	Tempo	Caráter
I — O painel de atuadores	1 a 8	50 min	Núcleo
II — C para o PIC	9 e 10	20 min	Núcleo — leitura em bancada
III — Uma nota é uma frequência	11 a 18	60 min	Núcleo
Anexos A e B	—	—	Opcional, para quem terminar antes

1 Antes de ligar a bancada

BANCADA

Por bancada: kit XM118 com fonte, cabo USB, computador com MPLAB X e XC8, aplicativo *Bootloader PIC18/XM118* v2.8, cronômetro, celular com aplicativo de afinador cromático (há vários gratuitos).

Gravação: o MPLAB X compila em modo *No Tool* e não grava. Pressione **SW9** — não SW1, como diz o manual — e, dentro de 4 a 5 segundos, envie o `.hex` pelo aplicativo do bootloader. Se a janela passar, repita.

2 Previsão

Esta tabela foi aberta há trinta minutos, no fim da aula teórica. Tenha-a à mão: o que interessa é o confronto entre o previsto e o observado.

PREVISÃO — ANTES DE ENERGIZAR

Previsão errada não é problema. Previsão ausente é: sem ela, o experimento vira demonstração. O que se avalia ao longo do semestre é a qualidade do raciocínio, nunca o acerto.

Quem chegou sem a folha preenche agora, antes de compilar qualquer coisa, e sem consultar a bancada ao lado. Uma frase por linha basta.

Pergunta	Previsão e justificativa	Con- fere?
P1. O que acontece com o pino no instante do <i>reset</i> , antes de a primeira linha executar?		
P2. Se <code>TRIS</code> for configurado antes de <code>LAT</code> , o comportamento muda? Como?		
P3. Com <code>LATC2 = 1</code> fixo, o buzzer emite som contínuo?		
P4. Num sinal quadrado, mudar a frequência afeta o quê no buzzer? E no color?		
P5. E mudar a proporção entre ligado e desligado?		
P6. LA4 vale 440 Hz e LA5 vale 880 Hz. Qual das duas exige que o programa inverta o pino mais vezes por segundo? Quantas vezes mais?		
P7. O laço de atraso é <code>while (n--) { NOP(); }</code> . Quanto tempo custa uma iteração — e como descobrir isso sem abrir o manual do compilador?		

P8. Uma variável `uint16_t` guarda de 0 a 65535. Quanto vale `c` depois de `c = 800 * 500; ?`

PARTE I

O painel de atuadores

3 A placa não tem pinos para todos

O PIC18F4550 tem 35 pinos de entrada e saída. A XM118 tem bem mais periféricos do que isso, e a solução do fabricante foi rotear vários deles para o **mesmo** pino, deixando a escolha para um banco de chaves DIP.

A chave CH3 comanda os atuadores do sistema de controle:

Chave	Atuador	Pino	Observação
CH3-2	SPEED	RC0	Realimentação do tacógrafo — é entrada, não saída
CH3-3	HEATER	RC1	Resistência de aquecimento; entra no Roteiro 6
CH3-4	LAMP	RC1	Lâmpada de 12 V — usada nesta oficina
CH3-5	COOLER	RC2	Ventoinha — usada na rodada B
CH3-6	BUZZER	RC2	Buzzer — usado na rodada A e na Parte III
CH3-7	DAC IN	RC2	Mantenha desligada o semestre inteiro

CONCEITO

Leia a tabela pelas colunas de pino, não pelas de atuador. **RC1** aceita o aquecedor ou a lâmpada; **RC2** aceita o cooler, o buzzer ou o DAC. Nunca dois ao mesmo tempo.

Isso tem uma consequência que organiza a sessão inteira: `LATCbits.LATC2 = 1` não significa “ligar o cooler” nem “ligar o buzzer”. Significa apenas **colocar o pino RC2 em nível alto**. O que acontece no mundo depende de uma chave mecânica que o programa não lê, não controla e não tem como consultar.

Por isso, na Parte I, os nomes no código são de **pino**, não de atuador.

CHAVES

Configuração inicial — rodada A. Confira uma a uma antes de energizar:

CH3-2	CH3-3	CH3-4	CH3-5	CH3-6	CH3-7
OFF	OFF	ON	OFF	ON	OFF
SPEED	HEATER	LAMP	COOLER	BUZZER	DAC

Ou seja: lâmpada em RC1, buzzer em RC2. Aquecedor e cooler desconectados.

4 O programa base

```
/* -----
Oficina 1 — um pino, dois significados
Os nomes abaixo são de PINO, não de atuador: quem decide o
atuador e a chave CH3, que o programa não consegue ler.
----- */
```

```
#define _XTAL_FREQ 16000000UL /* nucleo a 16 MHz – ver Aula 1 */

#include <xc.h>
#include <stdint.h>

#define LEDS_LAT LATD /* barra de 8 LEDs – pinos fixos */
#define LEDS_TRIS TRISD

#define RC1_LAT LATCbits.LATC1 /* HEATER (CH3-3) ou LAMP (CH3-4) */
#define RC1_TRIS TRISCbits.TRISC1

#define RC2_LAT LATCbits.LATC2 /* COOLER (CH3-5), BUZZER (CH3-6) */
#define RC2_TRIS TRISCbits.TRISC2 /* ou DAC (CH3-7) */

static void saidas_init(void)
{
    /* 1) estado seguro PRIMEIRO: tudo desligado */
    LEDS_LAT = 0x00;
    RC1_LAT = 0;
    RC2_LAT = 0;

    /* 2) so entao habilitar as saidas */
    LEDS_TRIS = 0x00;
    RC1_TRIS = 0;
    RC2_TRIS = 0;
}

void main(void)
{
    saidas_init();

    while (1) {
        /* os experimentos entram aqui, um de cada vez */
    }
}
```

NOTA

Não há bloco `#pragma config` neste programa, e isso é intencional: na XM118 quem fixa as palavras de configuração é o bootloader. Diretivas escritas na aplicação compilam, não reclamam e não têm efeito.

Se `_XTAL_FREQ` estiver em `48000000UL`, todos os atrasos sairão **três vezes mais longos** — e essa é a primeira coisa a verificar se algum tempo medido não bater.

5 Rodada A — lâmpada e buzzer

5.1 Os LEDs, com cronômetro

TAREFA

```
while (1) {
    LEDS_LAT = 0xFF;
    __delay_ms(500);
    LEDS_LAT = 0x00;
```

```
    __delay_ms(500);  
}
```

Cronometre **20 piscadas completas** e divida por 20. Se o período não bater com 1 s dentro de uns poucos por cento, `_XTAL_FREQ` está errado.

Vocês já piscaram LEDs por conta própria na sessão passada. A diferença aqui é a medição: sem cronômetro, “piscou” e “piscou no ritmo certo” são a mesma observação.

5.2 A lâmpada

TAREFA

```
RC1_LAT = 1;  
__delay_ms(10000);  
RC1_LAT = 0;
```

Anote: quanto tempo o filamento leva para apagar completamente depois que `RC1_LAT` vai a zero?

EXPERIMENTO

A lâmpada não apaga no instante em que o pino vai a zero. Esse atraso entre **comandar** e **observar o efeito** reaparece, muito mais lento, quando o aquecedor da mesma linha RC1 precisar elevar a temperatura do sensor. É a razão de existir do controle com histerese no Roteiro 9.

5.3 O buzzer com nível constante

TAREFA

```
RC2_LAT = 1;  
__delay_ms(3000);  
RC2_LAT = 0;
```

Escute com atenção. Descreva exatamente o que ouviu, e confronte com a previsão P3.

CONCEITO

O que se ouve é um **tique** curto no instante da subida, e depois silêncio.

Há dois componentes vendidos com o mesmo nome e o mesmo aspecto. O buzzer **ativo** tem um oscilador dentro dele: aplique nível alto e ele apita, na frequência que o fabricante escolheu — você liga e desliga o som, não escolhe a nota. O buzzer **passivo** é apenas um transdutor, uma membrana piezoelétrica: converte em movimento a tensão que recebe.

O XM118 traz um buzzer **passivo**. Nível alto empurra a membrana para um lado; ela vai até lá, para, e fica parada — e uma membrana parada não move ar. O tique é o deslocamento único.

A diferença não aparece no *datasheet* do microcontrolador nem no código: aparece no ouvido, em um segundo de teste. E é dela que depende a Parte III inteira: com um buzzer ativo, **quem compõe a melodia seria o fabricante**, não o firmware.

TAREFA

Faça o buzzer emitir som de verdade.

```
/* ~2 kHz por 300 ms */
for (uint16_t i = 0; i < 600; i++) {
    RC2_LAT = 1;
    __delay_us(250);
    RC2_LAT = 0;
    __delay_us(250);
}
```

Em caso de divergência com a previsão, escreva ao lado o que faltava no seu raciocínio — não apague a previsão original.

ATENÇÃO

Doze bancadas com buzzer contínuo tornam a sala inutilizável. Pulsos de até 300 ms, com pausa entre eles. Isso vale também para a Parte III.

6 A ordem que importa: LAT antes de TRIS

EXPERIMENTO

Inverta os dois blocos de `saidas_init`:

```
static void saidas_init(void)
{
    RC2_TRIS = 0;      /* saída habilitada ANTES ... */
    RC1_TRIS = 0;
    LEDS_TRIS = 0x00;

    RC2_LAT = 0;       /* ... do estado ser definido */
    RC1_LAT = 0;
    LEDS_LAT = 0x00;
}
```

Grave e pressione o *reset* várias vezes seguidas, prestando atenção ao buzzer e à lâmpada. Depois volte à ordem original e repita.

CONCEITO

No *reset*, todo pino nasce como **entrada** e o conteúdo de `LAT` é **indefinido**. Enquanto o pino é entrada, o valor de `LAT` não chega ao mundo: alta impedância, nada acontece.

Escrever `TRIS = 0` habilita a saída — e nesse instante o pino passa a refletir o que estiver em `LAT`. Se `LAT` ainda não foi escrito, o atuador é comandado por lixo até a linha seguinte corrigir.

Daí a regra do semestre: **defina o estado seguro em LAT, depois habilite a saída em TRIS**. Com um LED o transitório é invisível. Com um buzzer é audível, e com um motor ou uma resistência de aquecimento é mecânico e térmico.

7 O mesmo sinal, dois significados

Aqui está o experimento central da Parte I, e ele exige **uma única troca de chave** no meio. O código não muda entre as duas rodadas — nem uma linha.

TAREFA

Substitua o laço principal pelo bloco abaixo. Os LEDs indicam qual rodada está em execução.

```
while (1) {
    /* Rodada 1 — 1 kHz, 50% (referencia) */
    LEDS_LAT = 0x01;
    for (uint16_t i = 0; i < 4000; i++) {
        RC2_LAT = 1; __delay_us(500);
        RC2_LAT = 0; __delay_us(500);
    }
    LEDS_LAT = 0x00; __delay_ms(1500);

    /* Rodada 2 — 2 kHz, 50% */
    LEDS_LAT = 0x02;
    for (uint16_t i = 0; i < 8000; i++) {
        RC2_LAT = 1; __delay_us(250);
        RC2_LAT = 0; __delay_us(250);
    }
    LEDS_LAT = 0x00; __delay_ms(1500);

    /* Rodada 3 — 5 kHz, 50% */
    LEDS_LAT = 0x04;
    for (uint16_t i = 0; i < 20000; i++) {
        RC2_LAT = 1; __delay_us(100);
        RC2_LAT = 0; __delay_us(100);
    }
    LEDS_LAT = 0x00; __delay_ms(1500);

    /* Rodada 4 — 1 kHz, 10% */
    LEDS_LAT = 0x08;
    for (uint16_t i = 0; i < 4000; i++) {
        RC2_LAT = 1; __delay_us(100);
        RC2_LAT = 0; __delay_us(900);
    }
    LEDS_LAT = 0x00; __delay_ms(1500);

    /* Rodada 5 — 1 kHz, 90% */
    LEDS_LAT = 0x10;
    for (uint16_t i = 0; i < 4000; i++) {
        RC2_LAT = 1; __delay_us(900);
        RC2_LAT = 0; __delay_us(100);
    }
    LEDS_LAT = 0x00; __delay_ms(1500);
}
```

EXPERIMENTO

Passagem 1 — buzzer. Com CH3-6 ON e CH3-5 OFF, ouça o ciclo inteiro duas vezes e preencha a coluna do buzzer.

Passagem 2 — cooler. Desligue a placa. Troque **apenas duas chaves**: CH3-6 para OFF, CH3-5 para ON. Religue. **Não regrave nada.** Observe o ciclo inteiro duas vezes e preencha a coluna do cooler.

Ro- dada	Frequên- cia	Pro- porção	Buzzer (som)	Cooler (velocidade)
1	1 kHz	50%		

2	2 kHz	50%
3	5 kHz	50%
4	1 kHz	10%
5	1 kHz	90%

TAREFA

Compare as rodadas em dois grupos e responda por escrito:

1. **Rodadas 1, 2 e 3** mantêm a proporção em 50% e variam a frequência. O que mudou no buzzer? O que mudou no cooler?
2. **Rodadas 1, 4 e 5** mantêm a frequência em 1 kHz e variam a proporção. O que mudou no buzzer? O que mudou no cooler?
3. O microcontrolador executou exatamente o mesmo programa nas duas passagens. O que, então, decidiu o significado do sinal?

CONCEITO

O resultado esperado é este:

	Muda a frequência	Muda a proporção
Buzzer	o tom muda	o tom não muda
Cooler	a velocidade não muda	a velocidade muda

Frequência e proporção são dois parâmetros **independentes** do mesmo sinal, e cada carga responde a um deles. O buzzer converte a alternância em som e ignora quanto tempo o sinal passa em cada nível. O cooler tem inércia mecânica demais para seguir uma alternância de milissegundos: ele integra o sinal e responde à **energia média**, isto é, à proporção.

E o programa não sabe de nada disso. Ele coloca RC2 em nível alto e baixo em ritmos diferentes; o significado físico foi decidido por uma chave DIP.

NOTA

Guardem os nomes, porque a partir do Roteiro 3 eles serão usados o tempo todo: a proporção entre ligado e desligado chama-se **ciclo de trabalho** ou **razão cíclica** (*duty cycle*), e o par frequência-fixa-com-ciclo-variável chama-se **modulação por largura de pulso** — PWM.

Trocar os dois é o erro mais comum do Roteiro 3. Vocês acabaram de separá-los por observação, o que é bem mais difícil de esquecer que uma definição.

8 O que fica para o Roteiro 3

TAREFA

Ainda com o cooler conectado, responda:

1. Para mudar a velocidade do cooler neste programa, basta alterar um número em tempo de execução, ou é preciso reescrever e regravar?
2. Enquanto o laço da rodada 5 executa, o que mais o microcontrolador está fazendo?
3. Some os atrasos: quanto tempo de processador sobra, por segundo, para ler um sensor de temperatura e decidir alguma coisa?

CONCEITO

As respostas convergem. O laço ocupa **cem por cento** do tempo do processador: não sobra ciclo para ler o sensor, atualizar um display ou responder a um botão. E a proporção está congelada dentro de chamadas de `__delay_us`, isto é, no código compilado — não é uma variável que um controlador possa ajustar.

Um termostato precisa exatamente das duas coisas que faltam: variar a proporção continuamente, em função da temperatura lida, e fazer isso **sem** parar de executar o resto do programa.

É esse o problema que o Roteiro 3 resolve, transferindo a geração do sinal para um periférico dedicado que trabalha sozinho enquanto a CPU cuida de outra coisa. O módulo chama-se CCP1 — e ele vive justamente no RC2, o pino desta oficina. Quando ele assumir, o buzzer terá de sair da CH3-6 em definitivo.

CHAVES

Antes de seguir para a Parte III, volte à configuração da rodada A: CH3-6 ON (buzzer), CH3-5 OFF (cooler). O resto da oficina é no buzzer.

PARTE II

C para o PIC: o que muda em relação a Algoritmos

Você sabem escrever algoritmos. O que muda daqui em diante não é a lógica: é o que está por baixo dela. Dois kilobytes de RAM, uma unidade aritmética de oito bits, nenhuma unidade de ponto flutuante e nenhum sistema operacional para avisar quando algo der errado.

CONCEITO

A diferença que organiza esta parte: no computador, um erro de tipo ou de faixa costuma virar uma exceção, uma mensagem, um travamento. Aqui **quase nenhum** dos erros desta seção produz mensagem. Todos produzem **comportamento** — uma nota curta demais, um atraso três vezes maior, um valor que dá voltas.

Diagnosticar firmware é, em boa parte, reconhecer sintomas.

9 Sete coisas que o C do PIC cobra

9.1 Tipos com largura declarada

Escreva sempre a largura. `#include <stdint.h>` dá os nomes:

Tipo	Faixa	Bytes	Quando usar
<code>uint8_t</code>	0 a 255	1	Máscaras, contadores curtos, portas
<code>int8_t</code>	-128 a 127	1	Diferenças pequenas com sinal
<code>uint16_t</code>	0 a 65 535	2	Meio-período em μs , contagens de ciclos
<code>int16_t</code>	-32 768 a 32 767	2	Temperatura em décimos de °C (Roteiro 4)
<code>uint32_t</code>	0 a $\approx 4,3 \cdot 10^9$	4	Só em contas intermediárias

ATENÇÃO

No XC8 para PIC18, `int` tem **16 bits** — no seu computador, tem 32. Escrever `int` esconde a largura e transporta suposições erradas de uma máquina para outra. `uint32_t` custa quatro bytes de RAM e várias instruções por operação: não é a escolha padrão, é a exceção.

9.2 A divisão trunca e o estouro é silencioso

```
uint16_t a = 800, b = 500;
uint16_t c = a * b;      /* 400000 nao cabe em 16 bits */
```

CONCEITO

O valor 400 000 não cabe em 16 bits, e o C não reclama: ele descarta os bits que sobraram. Sobra o resto da divisão por 65 536, isto é, **6784** — quase dois ordens de grandeza abaixo do esperado, sem uma linha de aviso.

A promoção automática do C não salva ninguém aqui, porque o tipo para o qual ela promove — `int` — também tem 16 bits nesta máquina.

Regra prática: quando o produto de dois valores puder passar de 65 535, promova explicitamente o **primeiro** operando e faça a conta inteira em 32 bits, convertendo de volta só no fim:

```
c = (uint16_t)(((uint32_t)a * 500UL) / b);
```

E lembre que a divisão inteira trunca: $7 / 2$ vale 3, não 3,5. Para arredondar em vez de truncar, some metade do divisor antes: $(x + y/2) / y$.

Confira agora a sua previsão P8.

9.3 Ponto flutuante: não**NOTA**

O PIC18F4550 não tem hardware de ponto flutuante. Cada operação com `float` vira uma chamada de biblioteca — dezenas de microssegundos e centenas de bytes de Flash por operação. Na licença gratuita do XC8, ainda por cima sem otimização.

A saída da disciplina é **escalar**: em vez de guardar 23,5 °C num `float`, guarde 235 décimos de grau num `int16_t`. É a decisão que atravessa todos os roteiros a partir do 4, e ela vale a pena conhecer desde já.

9.4 #define é substituição textual, não variável

```
#define DECIMOS_US_POR_ITER 25u    /* nao ocupa um byte de RAM */
#define BUZZER_LAT LATCbits.LATC2  /* apelido de registrador  */
```

ATENÇÃO

O pré-processador troca texto por texto, antes de o compilador ver qualquer coisa. Daí a regra dos parênteses em macros com argumento:

```
#define MEIO(x)  x / 2    /* errado */
#define MEIO(x) ((x) / 2) /* certo  */
```

Com a primeira versão, `MEIO(4 + 4)` vira `4 + 4 / 2`, que vale 6.

9.5 const mora na Flash; sizeof custa zero

```
static const uint16_t MELODIA[] = { MI4, MI4, FA4, SOL4 };

#define N_NOTAS (sizeof(MELODIA) / sizeof(MELODIA[0]))
```

CONCEITO

Um vetor declarado `const` vai para a memória de **programa** — os 32 KB de Flash — e não para os 2 KB de RAM. Foi por isso que a arquitetura previu ler a memória de programa como dado (§5 da aula). Melodias, curvas de linearização de sensor e cadeias de texto vivem lá sem tocar na RAM.

`sizeof` é resolvido em tempo de compilação: `N_NOTAS` não é uma conta que o processador faça, é um número que o compilador escreve. Contar os elementos assim, em vez de digitar `15`, significa que acrescentar uma nota à melodia não exige lembrar de corrigir mais nada.

9.6 `static` e a ordem das funções

NOTA

`static` antes de uma função significa **visível só neste arquivo**. Antes de uma variável local, significa outra coisa: a variável não morre no fim da chamada, ela persiste entre chamadas.

E o C lê o arquivo de cima para baixo: uma função só pode ser chamada depois de o compilador já saber que ela existe. Defina as funções auxiliares **antes** de `main()`, ou declare o protótipo no topo. O sintoma de esquecer isso é o aviso *implicit declaration of function*.

9.7 Registradores são variáveis com nome

CONCEITO

Não há ponteiros nesta disciplina para chegar ao hardware: o `xc.h` já declara os nomes, e eles se usam como qualquer outra variável.

```
LATC = 0x04;           /* escreve os oito bits de PORTC de uma vez */
LATCbits.LATC2 = 1;    /* escreve apenas o bit 2 */
```

A segunda forma parece mais segura, e é — desde que o alvo seja `LAT`. Escrever em `PORT` obriga o processador a **ler** os pinos antes de alterar um bit, e o que ele lê é a tensão que estiver lá, não o que o programa mandou. Um pino carregado por uma carga pesada pode ser lido como zero logo depois de ter sido escrito como um, e a escrita seguinte propaga o engano para os vizinhos. **Escreva sempre em `LAT`**.

10 Sintomas de compilação e de bancada

Guarde esta tabela: ela cobre quase tudo o que vai dar errado hoje.

Sintoma	Causa	O que fazer
Erro citando <code>_XTAL_FREQ</code> ao usar <code>__delay_ms</code>	A macro precisa saber o clock e foi definida depois do <code>#include</code> , ou não foi definida	<code>#define _XTAL_FREQ</code> antes do <code>16000000UL</code> <code>#include <xc.h></code>
Todos os tempos saem três vezes maiores que o esperado	<code>_XTAL_FREQ</code> em <code>48000000UL</code> com o núcleo a 16 MHz	Corrigir a constante; é sempre a primeira hipótese
Erro na linha de <code>__delay_us(x)</code> com <code>x</code> variável	A macro exige constante de tempo de compilação	Escrever um laço de espera próprio — é a §13
<i>implicit declaration of function</i>	Função chamada antes de existir para o compilador	Mover a definição para cima de <code>main()</code>
Compila, grava, e nada acontece no pino	<code>TRIS</code> esquecido: o pino continua entrada	Conferir <code>saidas_init()</code>
Estalo no buzzer a cada <i>reset</i>	<code>TRIS</code> escrito antes de <code>LAT</code>	Inverter a ordem — §6

O atraso simplesmente desaparece	Laço sem efeito observável, eliminado pelo compilador	NOP () no corpo — §13
<i>Advisory: free license, optimizations disabled</i>	Comportamento normal da licença gratuita	Ignorar — mas ver §16.2, porque afeta a calibração

DIVERGÊNCIA — VERIFICAR ANTES DO USO

As mensagens exatas variam com a versão do XC8 instalada na bancada. Se a que aparecer na sua tela não bater com nenhuma linha desta tabela, anote-a literalmente no registro: a tabela é corrigida com o que vocês encontrarem.

PARTE III

Uma nota é uma frequência

11 Da nota ao número

Três fatos bastam para o dia de hoje.

- Por convenção internacional, a nota LA4 vale **440 Hz**.
- Subir uma oitava é **dobrar** a frequência: LA5 = 880 Hz.
- A oitava é dividida em doze semitons de razão constante $2^{1/12} \approx 1,05946$, de modo que

$$f(n) = 440 \cdot 2^{n/12}$$

onde n é a distância em semitons até o LA4, positiva para cima.

O microcontrolador, porém, não sabe o que é hertz. Ele sabe pôr o pino em alto, esperar, pôr em baixo, esperar. O que o programa precisa conhecer é **quanto tempo esperar entre duas inversões** — ou seja, o meio-período:

$$T_{1/2}[\mu s] = \frac{10^6}{2f}$$

Nota	DO4	RE4	MI4	FA4	SOL4	LA4	SI4	DO5
f (Hz)	261,63	293,66	329,63	349,23	392,00	440,00	493,88	523,25
$T_{1/2}$ (μs)	1911	1703	1517	1432	1276	1136	1012	956

TAREFA

Recalcule à **mão** duas linhas dessa tabela — a do LA4 e a de uma nota à sua escolha — e confira o arredondamento. É a única forma de a tabela deixar de ser um bloco de números copiados.

Confronte também a previsão P6: quantas inversões de pino por segundo o programa executa para tocar LA4? E para LA5?

12 Período e razão cíclica, agora com nome

Vocês já separaram as duas grandezas por observação na §7, com o buzzer e o cooler. Aqui elas ganham nome, fórmula e uma explicação de por que a separação existe.

CONCEITO

O **período** T é o intervalo após o qual o padrão se repete. É ele que determina **qual nota** soa.

A **razão cíclica** D é a fração do período em que o sinal está em nível alto. Ela não altera a nota — altera **como a nota soa**, isto é, o timbre.

Grandeza	Determina	No buzzer é...
Período T	a repetição do padrão	a informação: a nota
Razão cíclica D	o formato dentro do período	o timbre e o volume

NOTA

A tabela da §7 é a mesma coisa dita de outro jeito, e ela já anunciava a inversão de papéis que vem por aí. No buzzer, a informação está no período e a razão cíclica é enfeite. No cooler, a razão cíclica é que carrega a informação — ela regula a potência entregue — e o período vira um detalhe de implementação. **O mesmo pino RC2, os mesmos dois números, significados opostos.**

13 O tempo não se declara — mede-se

Aqui está a primeira lição séria de firmware da disciplina, e ela é desconfortável.

ATENÇÃO

`__delay_us()` exige uma **constante de tempo de compilação**. A macro é expandida em uma sequência fixa de instruções, contada pelo compilador. Escrever `__delay_us(meio_perodo)` com uma variável não compila — e para tocar uma melodia a espera precisa depender da nota, escolhida em execução.

Não há saída: é preciso escrever o laço de espera à mão.

CONCEITO

E aí aparece o problema. Quantos ciclos custa uma iteração de `while (n--) { NOP(); }`? Quem decide isso é o **compilador**, ao escolher as instruções que vai gerar. Você não decide, o manual do XC8 não promete, e o valor muda quando o nível de otimização muda.

Consequência prática: o tempo de um laço de atraso é uma **grandeza experimental**. Ele se mede. Essa é a resposta à previsão P7.

NOTA

E por que o `NOP()` dentro do laço? Sem ele, o corpo do laço não produz nenhum efeito observável, e o compilador tem plena liberdade para eliminá-lo: o atraso simplesmente desapareceria. `NOP()` é uma instrução que o compilador é obrigado a emitir — um ciclo de máquina gasto de propósito.

14 Escrevendo o programa

O arquivo `musica_esqueleto.c` está na pasta da disciplina. Ele já traz tudo o que não é interessante — mapeamento de pinos, tabela de notas, `saidas_init()`, modo de calibração — e **quatro lacunas**, que são o trabalho de vocês.

```
/* musica_esqueleto.c — preencha as lacunas E1 a E4 ----- */

#define _XTAL_FREQ 16000000UL
#include <xc.h>
#include <stdint.h>

#define MODO_CALIBRACAO      1      /* 1 = tom contínuo, 0 = melodia */
#define DECIMOS_US_POR_ITER 25u    /* <<< AJUSTAR APOS MEDIR (E2) */
#define UNIDADES_TESTE      400u

#define BUZZER_LAT           LATCbits.LATC2      /* CH3-6 ON */
```

```

#define BUZZER_TRIS TRISbits.TRISC2

#define D04 1911u /* ... tabela completa no arquivo ... */
#define PAUSA 0u

static const uint16_t MELODIA[] = { /* Ode a Alegria, 1a frase */ };
static const uint8_t DURACAO[] = { /* em unidades de 100 ms */ };
#define N_NOTAS (sizeof(MELODIA) / sizeof(MELODIA[0]))

/* ===== E1 ===== */
static void espera(uint16_t unidades)
{
    /* escreva aqui */
}

/* ===== E2 ===== */
static void tom_bruto(uint16_t unidades, uint16_t ciclos)
{
    /* escreva aqui */
}

/* ===== E3 ===== */
static void nota(uint16_t meio_periodo_us, uint16_t duracao_ms)
{
    /* escreva aqui */
}

static void saidas_init(void) { /* ja escrito - LAT antes de TRIS */ }

void main(void)
{
    saidas_init();
#if MODO_CALIBRACAO
    LEDS_LAT = 0xFF;
    while (1) { tom_bruto(UNIDADES_TESTE, 1000u); }
#else
    while (1) {
        /* ===== E4 ===== */
    }
#endif
}

```

14.1 E1 — o laço de espera

ESCREVA O CÓDIGO

Escreva `espera()`. São três linhas, e cada uma tem um motivo:

```

static void espera(uint16_t unidades)
{
    /* execute exatamente `unidades` iteracoes,
       com um NOP() no corpo */
}

```

Antes de compilar, responda no registro: quantas iterações executa `while (n--)` quando `n` vale 3? E quando vale 0? O que acontece com a variável `n` depois da última comparação — e por que isso não estraga nada aqui?

14.2 E2 — o tom, e a calibração do seu próprio laço

ESCREVA O CÓDIGO

Escreva `tom_bruto()`: ciclos períodos completos de onda quadrada com razão cíclica de 50%, usando `espera(unidades)` para cada metade.

```
static void tom_bruto(uint16_t unidades, uint16_t ciclos)
{
    /* enquanto houver ciclos:
       pino em alto, espera(unidades)
       pino em baixo, espera(unidades) */
}
```

Escreva no `LAT`, nunca no `PORT` (§9.7).

EXPERIMENTO

Calibrar o laço que você acabou de escrever.

1. Compile com `MOD0_CALIBRACAO` em 1 e grave. O programa toca um tom contínuo com `UNIDADES_TESTE = 400` iterações por meio-período.
2. Meça a frequência resultante f . O afinador do celular resolve; o osciloscópio em RC2 também — mas leia de novo o quadro de perigo da §1.
3. Calcule o custo de uma iteração, em décimos de microssegundo:

$$\text{DECIMOS_US_POR_ITER} = \frac{10^7}{2 \cdot f \cdot \text{UNIDADES_TESTE}}$$

Exemplo: se o afinador acusar 500 Hz, o resultado é 25 — ou seja, $2,5 \mu\text{s}$ por iteração, que a $T_{\text{cy}} = 250$ ns dão dez ciclos de máquina.

4. Substitua a constante no código e recompile com `MOD0_CALIBRACAO` em 0.

Anote o valor que a sua bancada produziu e compare com o de outra bancada. Se forem diferentes, descubra por quê antes de seguir.

NOTA

Vale reparar no que acabou de acontecer: vocês mediram o custo em ciclos de um trecho de código compilado, sem osciloscópio e sem contar instruções na listagem, usando um aplicativo gratuito e o ouvido. A Aula 1 forneceu a única ferramenta necessária — a relação entre T_{cy} e frequência.

Um detalhe honesto: a medida não isola a iteração. Ela inclui também a escrita no pino e o teste do laço externo, diluídos por 400 iterações. A constante encontrada vale bem perto do ponto de calibração e vai errando à medida que `unidades` diminui — isto é, nas notas agudas. Quem quiser fechar essa lacuna: calibre com `UNIDADES_TESTE` em 400 e depois em 100, e resolva o sistema de duas equações para separar o custo **por iteração** do custo **fixo por meio-período**.

14.3 E3 — de microssegundos para iterações

Esta é a lacuna onde a Parte II é cobrada. Para cada nota, o programa precisa converter o meio-período em número de iterações, e a duração pedida em número de períodos completos:

$$\text{unidades} = \frac{\text{meio_periodo_us} \cdot 10}{\text{DECIMOS_US_POR_ITER}}$$

$$\text{ciclos} = \frac{\text{duracao_ms} \cdot 500}{\text{meio_periodo_us}}$$

TAREFA

Antes de escrever uma linha, preencha à mão, sabendo que a nota mais grave da tabela tem meio-período de 1911 μs e que a nota mais longa da melodia dura 800 ms:

Produto intermediário	Maior valor	Cabe em uint16_t ?
<code>meio_periodo_us * 10</code>		
<code>duracao_ms * 500</code>		

ESCREVA O CÓDIGO

Agora escreva `nota()`. Ela precisa:

1. tratar PAUSA (meio-período igual a zero) como silêncio de `duracao_ms`, com o pino em nível baixo;
2. calcular `unidades` e `ciclos` pelas fórmulas acima, **com a proteção contra estouro discutida na §9.2**;
3. garantir que nenhum dos dois seja zero — se for, forçar 1;
4. chamar `tom_bruto()` e deixar o pino em nível baixo ao sair.

Faça de propósito: escreva primeiro a versão **sem** as conversões para 32 bits, grave e ouça. Descreva o som no registro. Depois corrija e ouça de novo.

CONCEITO

O sintoma do estouro é característico: a melodia toca **no tom certo e na velocidade errada**, como se alguém tivesse apertado o avanço rápido. A razão é que `duracao_ms * 500` deu a volta e `ciclos` ficou minúsculo — o meio-período, que define a nota, não foi afetado.

Repare no que isso ensina sobre diagnóstico: **o sintoma aponta a variável**. Tom certo e duração errada isola o problema na conta de `ciclos` e absolve a de `unidades`, antes de olhar uma linha de código.

Vale a pergunta inversa: a expressão de `unidades` sobreviveu sem a conversão. Até que frequência mínima ela sobreviveria? Some 65 535 dividido por 10 e converta de volta para hertz.

14.4 E4 — a melodia**ESCREVA O CÓDIGO**

Escreva o laço principal. Ele percorre as duas tabelas em paralelo e toca cada nota pela duração correspondente. Três exigências:

1. a duração da nota *i* é `DURACAO[i] * 100` milissegundos — cuidado com o tipo do produto;
2. entre uma nota e a seguinte, um silêncio curto de 15 ms;
3. os LEDs acompanham a nota em execução. Essa linha é dada, porque usa deslocamento de bits, que é conteúdo do Roteiro 2:

```
LEDS_LAT = (uint8_t)(1u << (i & 0x07u));
```


EXPERIMENTO

A **melodia**. Compile com `MODO_CALIBRACAO` em `0` e execute.

1. A melodia está reconhecível? Se estiver desafinada de forma **consistente** — tudo agudo ou tudo grave — o problema é a calibração, não o código.
2. **Remova a pausa de 15 ms**, recompile e ouça. Explique o que aconteceu com as notas repetidas, e por quê.
3. Substitua a melodia por outra, de sua escolha, escrevendo a tabela de meios-períodos a partir da fórmula da §11.

15 Por que 50% soa melhor

Decompondo o pulso retangular de amplitude A e razão cíclica D em série de Fourier, a amplitude do harmônico de ordem n vale

$$V_n = \frac{2A}{n\pi} \cdot |\sin(n\pi D)|$$

Dessa única expressão saem as duas consequências que interessam.

1. **A fundamental é máxima exatamente em $D = 0,5$** . Para $n = 1$, a amplitude é proporcional a $\sin(\pi D)$, que atinge o máximo em meio período. Em $D = 0,25$ a fundamental cai cerca de 3 dB; em $D = 0,1$, cerca de 10 dB. A nota fica mais fraca e menos definida.
2. **Em $D = 0,5$ todos os harmônicos pares desaparecem**, porque $\sin(n\pi/2) = 0$ para n par. Sobra apenas a série ímpar — o timbre “oco”, parecido com o de uma clarineta. Fora de 50%, o segundo harmônico reaparece; o ouvido o interpreta como uma oitava acima misturada à nota, e o som soa anasalado.

ATENÇÃO

“Melhor” aqui é **convenção**, não lei da física. Os videogames da geração do NES usavam 12,5%, 25% e 50% deliberadamente, como três timbres distintos. Meio período é o padrão porque maximiza a fundamental e produz o espectro mais limpo — não porque as outras razões sejam defeituosas. Razão cíclica é um **parâmetro de projeto**, e vocês vão voltar a escolhê-la por motivos bem diferentes quando o assunto for potência.

ESCREVA O CÓDIGO

E5 — ouvir o espectro. Não é preciso periférico nenhum para variar a razão cíclica: basta quebrar a simetria do laço. Escreva uma variante:

```
static void tom_assimetrico(uint16_t a, uint16_t b, uint16_t ciclos)
{
    while (ciclos--) {
        BUZZER_LAT = 1; espera(a);
        BUZZER_LAT = 0; espera(b);
    }
}
```

Escolha uma nota e mantenha $a + b$ **fixo**. Toque três versões e preencha a coluna da direita *antes* de compilar:

Versão	D aproximado	Previsão antes de ouvir
$a = b$	50%	
$a = b/3$	25%	

$a = b/7$	12,5%	
-----------	-------	--

Pergunta de fechamento: a nota mudou entre as três versões? Por quê?

16 Três limites que vocês acabaram de encontrar

Nada do que segue é defeito do programa de vocês. São propriedades da técnica.

16.1 A CPU está inteiramente ocupada

EXPERIMENTO

Repare que a animação dos LEDs só muda **entre** as notas. Durante a execução de uma nota, o microcontrolador não faz absolutamente mais nada — está preso contando iterações.

Acrescente ao laço da melodia a leitura de uma chave que acenda um LED extra enquanto pressionada. Tente usá-la durante uma nota longa. Descreva o comportamento.

16.2 A afinação é frágil

EXPERIMENTO

Recompile o mesmo código com um nível de otimização diferente no XC8. Ouça. Explique, em uma frase, por que a melodia desafinou sem que uma linha do programa fosse alterada.

16.3 Há tremulação

EXPERIMENTO

Se houver osciloscópio disponível, observe o período do sinal em RC2 ao longo de vários ciclos. Cada iteração custa “quase” o mesmo tempo, não exatamente o mesmo. Registre a variação observada.

17 Onde isso encosta no seu curso

CONCEITO

O que vocês montaram nesta oficina é, do ponto de vista de circuitos, um **inversor monofásico** elementar: uma chave que alterna uma carga entre dois níveis, produzindo uma onda retangular cujo conteúdo harmônico é dado exatamente pela expressão da §15.

O que o músico chama de **timbre**, o engenheiro de potência chama de **distorção harmônica**. É a mesma medida, com nomes de disciplinas diferentes. E a razão cíclica que vocês variaram para mudar o timbre é a mesma grandeza que, em um inversor, regula a tensão entregue à carga.

18 O que fica em aberto

Os três limites da §16 têm uma causa comum: **o tempo está sendo medido pela própria CPU, contando instruções**. Enquanto ela conta, não faz mais nada, e a contagem depende do código que o compilador gerou.

O PIC18F4550 tem, dentro dele, hardware dedicado a contar tempo de forma independente do programa. Com ele, o processador é acionado apenas quando a nota precisa mudar, e fica livre no resto do intervalo. É o assunto das próximas aulas — e é também o que vai permitir, mais adiante, regular a potência do cooler pela razão cíclica em vez do período.

NOTA

Fica um aviso honesto para quem for adiantar leitura: o módulo do PIC18 dedicado a gerar sinais com razão cíclica programável — o CCP1, aquele mesmo que vive em RC2 — tem uma **frequência mínima** de cerca de 977 Hz nesta placa. As notas de hoje estão quase todas abaixo disso. Ele não é a resposta para o buzzer.

A generalização “use o periférico” não substitui verificar a faixa de operação: cada bloco de hardware tem uma, e ela está no *datasheet*.

TAREFA

Preparação para a próxima sessão — entregar por escrito, meia página.

Suponha um contador de hardware que incrementa a cada 250 ns e gera um aviso quando transborda. Você quer que o aviso ocorra a cada meio-período do LA4 (1136 μ s).

1. Quantas contagens são necessárias?
2. Se o contador for de 8 bits, o valor cabe? E se for de 16 bits?
3. Que vantagem essa técnica tem sobre o laço de hoje, em relação ao limite §16.1?

ANEXOS

Para quem terminou antes

NOTA

Nada nos dois anexos é necessário para os roteiros seguintes. O Anexo A é a **única** oportunidade do semestre de mexer nos relés; o Anexo B é o gancho para o Roteiro 2.

Anexo A — o relé, que não segue nada

CHAVES

Apenas CH5-1 ON (relé 1, em RC6).

CH5-3 e CH5-4 **devem permanecer OFF**. Elas comandam os relés 3 e 4, em RD6/RD7 — os bits 6 e 7 da mesma PORTD da barra de LEDs. Com elas ligadas, o `LEDS_LAT = 0xFF` da §5 comuta dois relés uma vez por segundo, indefinidamente.

NOTA

Os relés 1 e 2 estão em RC6/RC7, que a partir do Roteiro 8 pertencem à comunicação serial; os relés 3 e 4 estão em RD6/RD7, que a partir do Roteiro 4 pertencem ao LCD. Esta oficina é a única sessão em que os relés estão livres de conflito. Depois dela, CH5 fica desligada em definitivo.

O relé da placa é um **A1RC2**: bobina de 12 VDC, contatos de 10 A, três bornes. Três bornes significa comutação, não apenas acionamento:

Borne	Comportamento
COM	Comum — o ponto que é chaveado
NF	Ligado a COM com a bobina desenergizada
NA	Ligado a COM com a bobina energizada

CONCEITO

O relé é uma chave mecânica que o programa aciona. A diferença em relação a tudo o que foi visto até aqui é o **isolamento**: o circuito de 5 V do microcontrolador e o circuito chaveado não têm ligação elétrica nenhuma — a ligação é magnética, através da bobina. É por isso que um relé pode chavear tensões e correntes que destruiriam o microcontrolador.

E o estado de repouso importa: com a bobina desenergizada, COM está em NF. Um *reset* deixa a carga em NA desligada — que é mais uma instância da regra de definir o estado seguro antes de habilitar a saída.

PERIGO

- **Nada de 127 V ou 220 V nos contatos.** Os 10 A do relé permitem, a bancada aberta não. Use exclusivamente 12 V.
- Carga sugerida: lâmpada automotiva de 12 V e 5 W — cerca de 0,42 A, ou 4% da capacidade dos contatos.
- Carga indutiva (motor, solenoide) exige diodo de roda livre em antiparalelo. A lâmpada é resistiva e dispensa.

TAREFA

Acione o relé a 1 Hz e escute.

```

#define RC6_LAT    LATCbits.LATC6    /* rele 1 – CH5-1 */
#define RC6_TRIS   TRISCbits.TRISC6

/* acrescenta em saidas_init(), na ordem correta:
   RC6_LAT = 0;
   RC6_TRIS = 0; */

for (uint8_t i = 0; i < 20; i++) { /* 20 ciclos, não infinito */
    RC6_LAT = 1; __delay_ms(500);
    RC6_LAT = 0; __delay_ms(500);
}

```

Os dois cliques — o de fechar e o de abrir — soam iguais? Se houver lâmpada ligada em NA e outra em NF, as duas chegam a ficar acesas ao mesmo tempo?

EXPERIMENTO

Onde o relé deixa de acompanhar. Repita com períodos cada vez menores, sempre com número limitado de ciclos:

Frequência	Meio período	Ciclos	Acompanha?
1 Hz	__delay_ms(500)	20	
5 Hz	__delay_ms(100)	50	
10 Hz	__delay_ms(50)	50	
20 Hz	__delay_ms(25)	50	

Em alguma frequência a lâmpada deixa de piscar e passa a ficar num brilho intermediário, ou o clique vira um zumbido. Anote **qual**.

ATENÇÃO

Pare em 20 Hz. Não aplique ao relé as rodadas de 1 a 5 kHz da §7: a vida útil dos contatos é contada em número de operações, e alguns segundos a quilohertz consomem o equivalente a anos de uso normal. Pelo mesmo motivo, não tente tocar uma melodia no relé.

TAREFA

A partir da frequência em que o relé falhou, estime o tempo de comutação mecânica. Se ele acompanha até $f_{\max}$ mas não além, cada meio período em $f_{\max}$ é aproximadamente o tempo de que a armadura precisa para se mover.

Compare essa ordem de grandeza com $T_{cy} = 250$ ns. Quantas instruções o processador executa enquanto o relé fecha um contato?

CONCEITO

A terceira carta. A matriz da §7 ganha agora uma linha:

	Muda a frequência	Muda a proporção
Buzzer	o tom muda	nada muda
Cooler	nada muda	a velocidade muda
Relé	nada muda	nada muda

O relé não responde a nenhum dos dois parâmetros: ele só liga e desliga, e lentamente. É essa a resposta antecipada à pergunta que sempre aparece no Roteiro 3 — **por que não usar um relé em vez de PWM?** Porque um relé não tem meio-termo, e porque a frequência necessária para simular meio-termo o destrói.

Três cargas, o mesmo pino de saída digital, três comportamentos qualitativamente distintos. O programa é idêntico nos três casos.

Anexo B — oitavas de graça

A tabela da §11 cobre uma oitava. Estendê-la para o instrumento inteiro não exige tabelar mais nada.

TAREFA

Dobrar a frequência é subir uma oitava; dividir por dois é descer. Em termos de meio-período, é o contrário: o meio-período **cai pela metade** ao subir uma oitava.

Acrescente à melodia um parâmetro de oitava, de modo que a mesma tabela sirva para várias delas. Divisão e multiplicação por dois são deslocamentos de bits — $\gg 1$ e $\ll 1$ — e custam um ciclo, contra dezenas de uma divisão genérica.

Toque a mesma frase duas oitavas acima. Ela ainda soa afinada? Confronte com o aviso da §14 sobre a calibração valer perto do ponto medido.

NOTA

Deslocamento de bits é justamente o conteúdo do Roteiro 2. Quem quiser antecipar, o gancho está aqui.

Até onde isso vai: um arquivo MIDI de verdade não caberia — exigiria interpretar o formato, converter número de nota em frequência com potenciação e reservar memória para os eventos. A conversão de oitavas, porém, sai de graça, e é o que separa uma tabela de números de um instrumento.

Registro

Item	O que registrar
Previsões	A tabela da §2, preenchida antes dos experimentos, com as correções ao lado — sem apagar o original
Período medido	Tempo de 20 piscadas dividido por 20, e o desvio em relação a 1 s
Lâmpada	Tempo aproximado até o filamento apagar
Buzzer	O que se ouviu com nível constante, e com alternância
Ordem de inicialização	O que mudou entre as duas versões de <code>saidas_init</code>
Matriz	A tabela de cinco rodadas, com as duas colunas preenchidas
Respostas	As três perguntas da §7 e as três da §8
E1	Iterações de <code>while (n--)</code> para <code>n</code> igual a 3 e a 0
E2 — calibração	O valor de <code>DECIMOS_US_POR_ITER</code> da sua bancada, a frequência medida e o valor de outra bancada
E3 — estouro	A tabela dos produtos intermediários, e o som produzido pela versão sem conversão para 32 bits
E4	O que aconteceu ao remover a pausa de 15 ms, e por quê
E5 (opcional)	As três previsões de timbre e o que se ouviu
Limites	As respostas de §16.1, §16.2 e, se houver osciloscópio, §16.3

Anexo A (opcional)	A frequência em que o relé deixou de acompanhar, e o tempo de comutação estimado
Divergências	Qualquer discordância entre este roteiro, o manual da Exsto e a serigrafia — inclusive mensagens do compilador fora da tabela da §10

CHAVES

Antes de sair, devolva as chaves ao estado da rodada A: CH3-2 OFF, CH3-3 OFF, CH3-4 ON, CH3-5 OFF, CH3-6 ON, CH3-7 OFF.

E desligue a CH5 inteira, inclusive a CH5-1 usada no Anexo A. A partir do Roteiro 4 os relés colidem com o LCD, e a partir do Roteiro 8 com a serial — uma chave de CH5 esquecida ligada produz falhas difíceis de diagnosticar meses depois. A próxima turma começa daí.

SEM NOTA

Nada disso vale nota. O registro serve à discussão de abertura da próxima sessão e ao seu próprio proveito quando o Roteiro 3 chegar.